CRISPYx NB-GLM Variants: Default vs PyDESeq2-Parity Mode

CRISPYx Development Team

January 2026

Abstract

CRISPYx provides two modes for its Negative Binomial Generalized Linear Model (NB-GLM) differential expression analysis: a **default production mode** optimized for speed and robustness, and a **PyDESeq2-parity mode** designed for validation against PyDESeq2 results. This document details the key differences between these two modes, their performance characteristics, and recommendations for when to use each.

1 Introduction

The CRISPYx NB-GLM implementation offers flexibility in how size factors, dispersions, and standard errors are computed. The default settings are optimized for production use in large-scale CRISPR screen analysis, while the PyDESeq2-parity mode replicates the exact behavior of PyDESeq2 for validation purposes.

The two modes differ in three key parameters:

- **Size Factor Scope**: How size factors (normalization factors) are computed
- **Dispersion Scope**: How gene-wise dispersions are estimated (the key difference)
- **Standard Error Method**: How standard errors for LFC estimates are computed

2 Key Differences

3 Detailed Parameter Explanations

3.1 Size Factor Scope

Size factors are normalization constants that account for differences in sequencing depth across cells.

Table 1: Summary of differences between NB-GLM variants

Parameter	**NB-GLM (default)**	**NB-GLM (pydeseq2)**
Size Factor Scope	`global`	`per_comparison`
Dispersion Scope	`global`	`per_comparison`
SE Method	`sandwich` (robust)	`fisher` (standard)
Purpose	Production use	PyDESeq2 parity testing

Global (`size_factor_scope="global"`):

- Computes size factors **once** using all cells in the dataset
- Stored in `adata.obs["size_factors"]` and reused for all comparisons
- More efficient: avoids redundant computation
- Recommended for consistency across comparisons

Per-Comparison (`size_factor_scope="per_comparison"`):

- Computes size factors **separately** for each control + perturbation pair
- Matches PyDESeq2 behavior exactly, which creates a separate DESeq-DataSet for each comparison
- Size factors are specific to the cells included in each comparison
- Required for exact numerical parity with PyDESeq2

The mathematical formulation for size factors uses the median-of-ratios method:

$$s_j = \text{median}_i \frac{K_{ij}}{\left(\prod_{j'=1}^{n} K_{ij'}\right)^{1/n}} \tag{1}$$

where K_{ij} is the count for gene i in cell j, and n is the number of cells in the scope.

3.2 Dispersion Scope (Key Difference)

Dispersion parameters model the overdispersion in count data relative to a Poisson distribution. **This is the most impactful difference between the two variants.**

Global (`dispersion_scope="global"`):

- Uses `precompute_global_dispersion()` with `fast_mode=True`
- Estimation pipeline: MoM (Method of Moments) → Trend fitting
- Computes dispersions once using all cells
- Stored in `adata.var["dispersion"]` for reuse
- Produces **moderate shrinkage** of LFC estimates
- Dispersion range typically 0.0 – 10.0

Per-Comparison (`dispersion_scope="per_comparison"`):

- Estimates dispersions independently for each perturbation vs. control comparison
- Full estimation pipeline: MoM → Trend → MAP (Maximum a Posteriori)
- Matches PyDESeq2's per-comparison dispersion estimation
- Can produce **extreme dispersion values** (e.g., 0.07 – 966)
- Results in **stronger shrinkage** of LFC estimates

The dispersion trend is modeled as:

$$\alpha_{\text{trend}}(\mu) = \frac{a_0}{\mu} + a_\infty \tag{2}$$

where a_0 and a_∞ are fitted parameters.

3.2.1 Impact on Shrinkage Strength

Empirical analysis on the Adamson dataset shows significant differences in shrinkage behavior:

Table 2: Shrinkage strength comparison (shrunk/raw LFC ratio, lower = more shrinkage)

Perturbation	**Default**	**Pydeseq2**	**Difference**
AARS	0.595	0.532	Pydeseq2 shrinks 15.6% more
AMIGO3	0.630	0.359	Pydeseq2 shrinks 73.3% more

This difference in shrinkage strength explains why the pydeseq2 variant may show lower overlap with PyDESeq2's lfcShrink results—see Section 5.

3.3 Standard Error Method

The standard error method affects confidence intervals and p-values for LFC estimates.

Sandwich (`se_method="sandwich"`):

- Uses the robust sandwich (Huber-White) variance estimator
- Robust to model misspecification (e.g., incorrect dispersion estimates)
- Provides valid inference even when the NB model is slightly wrong
- Default for production use

The sandwich estimator is:

$$\mathrm{Var}(\hat{\beta})_{\text{sandwich}} = (X^T W X)^{-1} X^T \mathrm{diag}(r_i^2) X (X^T W X)^{-1} \tag{3}$$

where W is the weight matrix and r_i are the Pearson residuals.

Fisher (`se_method="fisher"`):

- Uses the Fisher information matrix (inverse of expected Hessian)
- Assumes the model is correctly specified
- Matches PyDESeq2's standard error computation exactly
- Required for exact numerical parity with PyDESeq2

The Fisher information estimator is:

$$\mathrm{Var}(\hat{\beta})_{\text{fisher}} = (X^T W X)^{-1} \tag{4}$$

4 Performance Comparison

We benchmarked both modes on the Adamson subset dataset (1,716 cells, 11,630 genes, 3 perturbations).

Table 3: Performance comparison on Adamson subset dataset

Metric	**NB-GLM (default)**	**NB-GLM (pydeseq2)**	**Ratio**
Runtime	∼20 s	∼36 s	1.8× slower
Peak Memory	1,400 MB	1,526 MB	1.09× more

4.1 Why Default is Faster

The default mode achieves better performance through:

1. **Global precomputation**: Size factors and dispersions are computed once and reused for all N perturbation comparisons, reducing redundant calculations.
2. **Fast dispersion mode**: Skips the MAP shrinkage step, using trend-fitted dispersions directly.
3. **No per-comparison subsetting**: Avoids creating separate data subsets for each comparison.

For a dataset with N perturbations:

- **Default**: $O(1)$ dispersion estimation + $O(N)$ GLM fitting
- **PyDESeq2-parity**: $O(N)$ dispersion estimation + $O(N)$ GLM fitting

The speedup becomes more pronounced as N increases.

5 The Overlap Paradox

Counterintuitively, the pydeseq2 variant sometimes shows **lower overlap** with PyDESeq2's lfcShrink results than the default variant. This is explained by:

1. **PyDESeq2 produces aberrant shrinkage**: As documented in `pydeseq2_shrinkage_aberration.tex`, PyDESeq2's apeGLM shrinkage can *inflate* LFC magnitudes (22.5% of genes for AARS).
2. **CRISPYx pydeseq2 variant shrinks correctly but more aggressively**: The per-comparison dispersion estimation produces stronger (but correct) shrinkage.
3. **Coincidental alignment**: The default variant's weaker shrinkage happens to produce values closer to PyDESeq2's aberrantly inflated LFCs.

Conclusion: Lower overlap with PyDESeq2 does not indicate worse quality—both CRISPYx variants produce correct shrinkage (0% inflation), while PyDESeq2's aberrant results coincidentally align better with weaker shrinkage.

6 Recommendations

6.1 Summary

- Use **NB-GLM (default)** for day-to-day CRISPR screen analysis. It is faster, uses less memory, and provides robust standard errors.
- Use **NB-GLM (pydeseq2)** when you need exact numerical agreement with PyDESeq2, such as for benchmarking or validating results against published analyses.

Table 4: When to use each NB-GLM variant

Use Case	Recommended Mode
Production analysis	NB-GLM (default)
Comparing with PyDESeq2 results	NB-GLM (pydeseq2)
Large datasets (many perturbations)	NB-GLM (default)
Replicating published PyDESeq2 analysis	NB-GLM (pydeseq2)
Robust inference with uncertain dispersions	NB-GLM (default)
Validation/testing against PyDESeq2	NB-GLM (pydeseq2)

7 Configuration Examples

7.1 Benchmarking Configuration

In the CRISPYx benchmarking framework, the two modes are configured as follows:

Listing 1: Benchmarking configuration for NB-GLM variants

```
# Default NB-GLM (implicit settings)
"crispyx_de_nb_glm": {}

# PyDESeq2-parity NB-GLM
"crispyx_de_nb_glm_pydeseq2": {
    "size_factor_scope": "per_comparison",
    "dispersion_scope": "per_comparison",
    "se_method": "fisher",
}
```

7.2 Python API Usage

Listing 2: Direct Python API usage

```
import crispyx as cx

# Default mode (recommended for production)
cx.de.nb_glm(
    adata,
    size_factor_scope="global",       # default
    dispersion_scope="global",        # default
    se_method="sandwich",             # default
)

# PyDESeq2-parity mode (for validation)
cx.de.nb_glm(
    adata,
    size_factor_scope="per_comparison",
```

```
    dispersion_scope="per_comparison",
    se_method="fisher",
)
```

8 Conclusion

CRISPYx provides two NB-GLM modes to serve different needs: a fast, robust default mode for production use, and a PyDESeq2-parity mode for validation and comparison. The default mode is approximately 1.8× faster and uses 9% less memory, making it ideal for large-scale CRISPR screen analysis. The key difference is the **dispersion scope**: global estimation produces moderate shrinkage while per-comparison estimation can produce stronger shrinkage due to more variable dispersion estimates.

The PyDESeq2-parity mode ensures methodological alignment with PyDESeq2, which is essential for benchmarking and reproducing published results—though as documented, PyDESeq2's lfcShrink can produce aberrant results, so exact numerical agreement is not always desirable.